

ECE 5755 Lab 1: Kernel Implementation and Top-Down Analysis

Intel Xeon Silver 4514Y (Sapphire Rapids); Level-2 profiling

1. Kernel implementations

Convolution.

convolution() computes a valid multi-channel 2-D convolution. It allocates one output image per filter, initializes each output with the filter bias, and accumulates products across channels and kernel coordinates. For input side N and kernel side K, the output side is N-K+1. The nested loops perform $O(F(N-K+1)^2 C K^2)$ work.

ReLU.

relu() returns x when x is positive and zero otherwise. applyRelu() applies this operation in place across the input array, giving linear work and avoiding a second output allocation.

Linear layer.

linear() implements a fully connected layer. Each output starts with its bias and accumulates a dot product between the input vector and one row of weights. Its work is $O(I O)$, where I and O are the input and output sizes.

Matrix multiplication.

matmul() checks that the left matrix column count equals the right matrix row count, allocates the result, and uses the conventional three-loop row-column-inner-product algorithm. It returns NULL for incompatible dimensions.

Softmax.

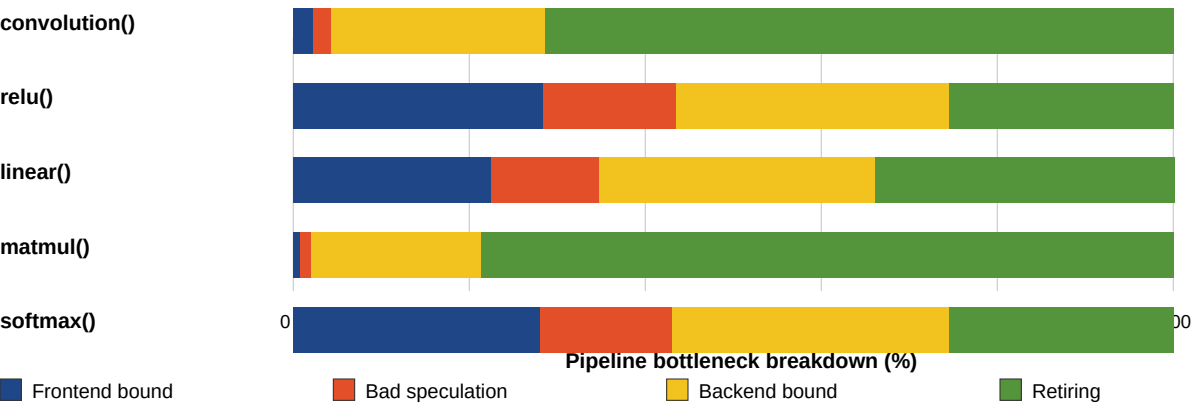
softmax() first finds the maximum input, subtracts it before exponentiation for numerical stability, sums the exponentials, and returns log-normalized probabilities. It uses temporary storage for exponentials and a separate output array.

2. Tests implemented

The Unity test driver covers convolution, neural-network helpers, functional operations, linear layers, and matrix operations. The tests include ordinary cases and boundary cases such as zero and negative values, large inputs and parameters, softmax normalization, ReLU behavior, square matrix multiplication, and incompatible matrix dimensions. These tests separate numerical correctness from performance measurement and expose indexing, initialization, bias, allocation, and dimension-validation errors.

3. Top-down analysis profile

Each standalone kernel was profiled with pmu-tools/toplev.py on an instructional compute node. The chart shows the Level-1 categories reported during Level-2 runs with input size 500. Every bar totals 100 percent.



Kernel	Frontend	Bad spec.	Backend	Retiring
convolution()	2.3%	2.0%	24.3%	71.4%
relu()	28.4%	15.1%	31.0%	25.5%
linear()	22.5%	12.3%	31.3%	34.0%
matmul()	0.8%	1.2%	19.3%	78.7%
softmax()	28.0%	15.1%	31.4%	25.5%

4. Profiling observations and bottlenecks

Convolution.

The profile is dominated by retiring (71.4%), with backend bound at 24.3%. The Level-2 breakdown is approximately 22.7% core bound and 1.6% memory bound. Explain how the filter/channel loop nest and arithmetic intensity produce this result. Add observations for other input sizes here.

ReLU.

ReLU has a balanced profile: 28.4% frontend bound, 15.1% bad speculation, 31.0% backend bound, and 25.5% retiring. Its Level-2 values include 17.1% fetch latency, 11.4% fetch bandwidth, 14.5% branch mispredicts, and 19.2% memory bound. Explain the effects of the conditional branch, input distribution, and memory access.

Linear.

The linear layer has 31.3% backend bound and 34.0% retiring. Its Level-2 breakdown is approximately 16.4% memory bound and 14.8% core bound, with 22.5% frontend bound. Relate the result to weight layout, cache behavior, and the dot-product loop order.

Matmul.

Matrix multiplication has the highest retiring fraction (78.7%) and small frontend and bad-speculation fractions. Its backend bound value is 19.3%, consisting of approximately 18.5% core bound and 0.8% memory bound. Explain the effect of regular control flow, arithmetic work, and locality in the naive loop order.

Softmax.

Softmax has a profile similar to ReLU: 28.0% frontend bound, 15.1% bad speculation, 31.4% backend bound, and 25.5% retiring. Its Level-2 breakdown includes 16.7% fetch latency, 11.4% fetch bandwidth, 14.5% branch mispredicts, and 19.2% memory bound. Discuss multiple passes, temporary arrays, and math-library operations.

5. Debugging process

Describe how compiler diagnostics, unit-test failures, targeted tests, memory checking, and profiling output were used to locate and correct bugs. Include one or two concrete failures and the corresponding fixes.

Student notes: _____

6. Performance improvements

The top-down data suggests reducing instruction and branch overhead for frontend-bound kernels, improving locality and contiguous allocation for backend-bound kernels, and using compiler vectorization where appropriate. ReLU may benefit from branchless or SIMD execution; linear and convolution may benefit from contiguous layouts and SIMD-friendly inner loops; matmul is a candidate for cache blocking and loop reordering; and softmax could reduce passes and temporary storage while preserving numerical stability.

Student notes: Describe the optimizations attempted or proposed and cite the top-down metric motivating each one. Do not claim an improvement without a measurement.

7. Profiling issues and source disclosure

Record issues such as login-node perf restrictions, Slurm time limits, CPU affinity, NMI watchdog warnings, counter multiplexing, small-input overhead, failed runs, and how they were resolved. List any external code, documentation, or AI assistance according to the course policy.

Student notes: _____
